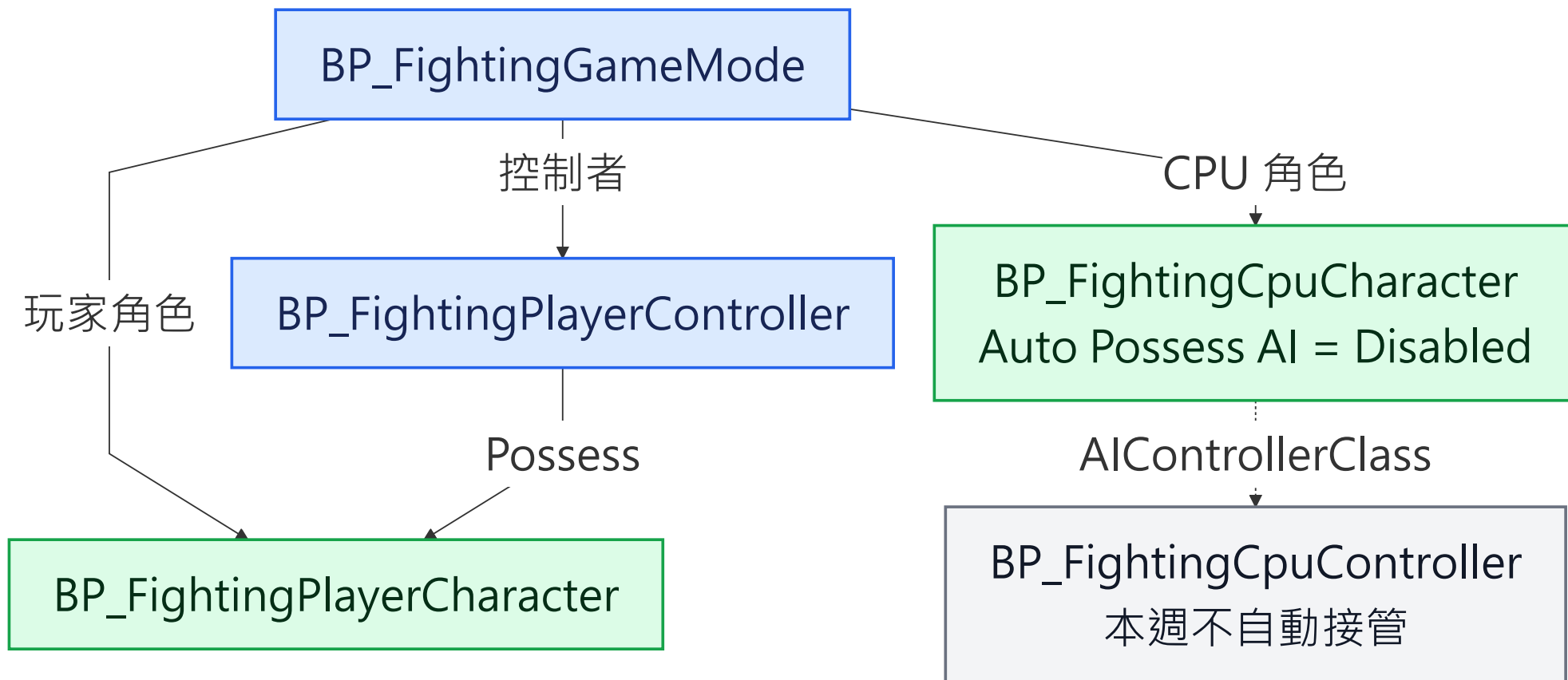
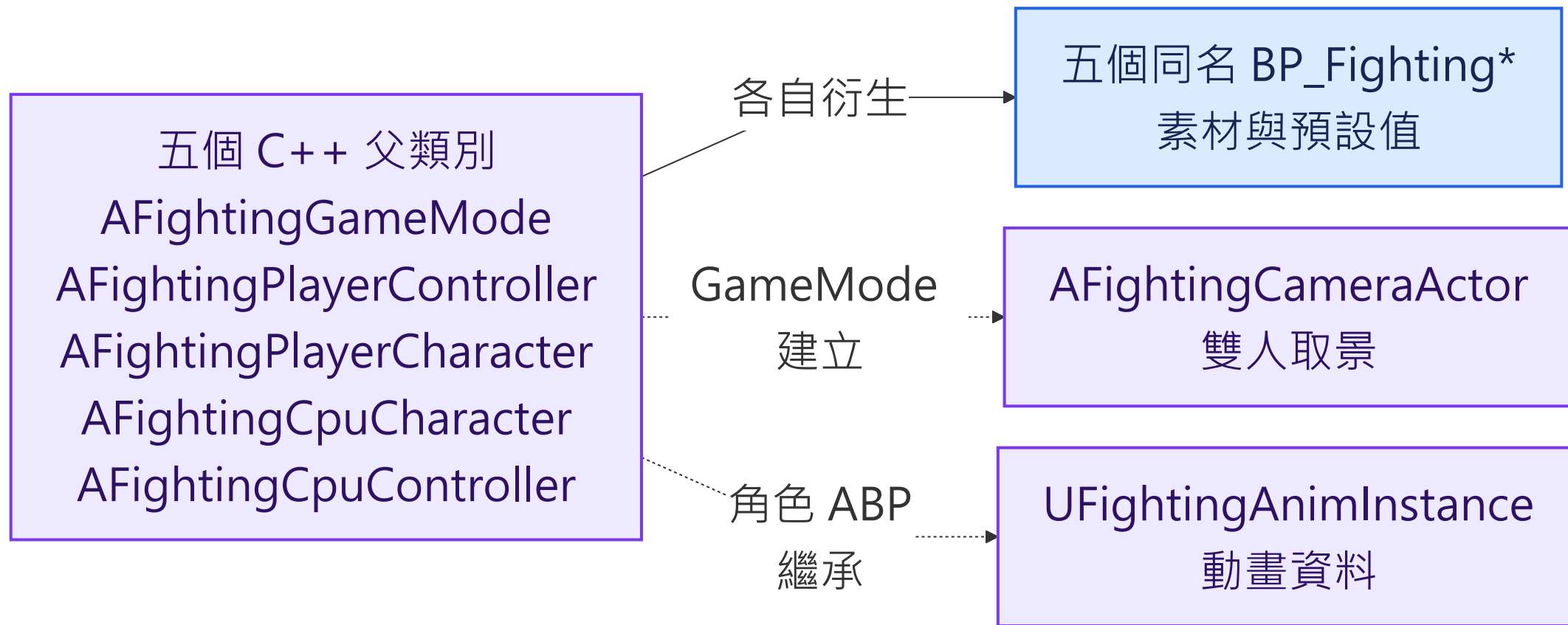


五個 BP：從對局設定認識角色與控制者



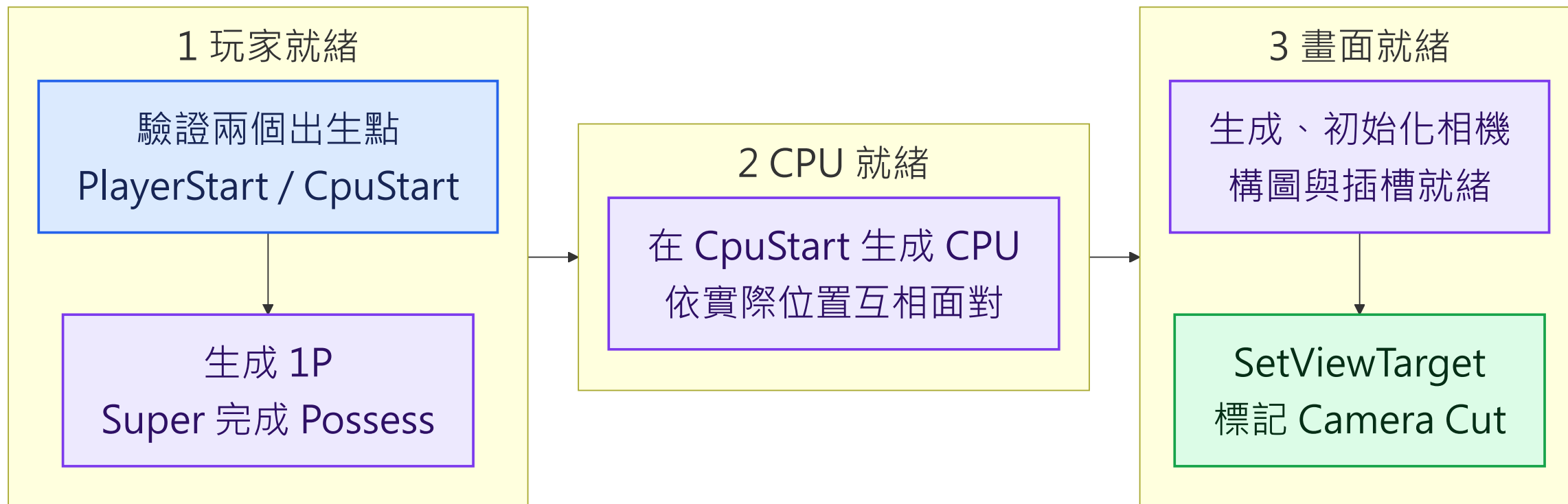
GameMode 選用角色與控制者，PlayerController 接管 1P；CPU 的 AI 控制目前關閉。

C++ 父類別與功能模組：誰負責哪一段



C++ 定義行為，BP 選素材與預設值，再由對局流程把它們接起來。

開場順序：玩家就緒後，才建立 CPU 與攝影機



先讓雙方角色定位，再完成相機構圖，最後才切換玩家畫面。

UObject、Actor、Component：繼承與擁有要分開看



Actor 與 Component 都是 UObject 子類；Actor 擁有 Component，把功能組合起來。

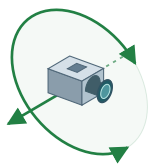
相機的三層元件：焦點、臂長、取景

② CameraBoom

支臂，決定方向與臂長

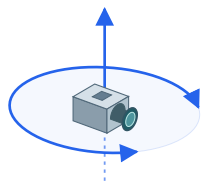
Spring Arm 是虛擬臂

Pitch / Y 軸



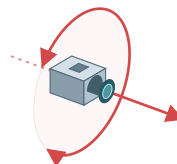
上下俯仰
預設 -10°

Yaw / Z 軸



左右轉向

Roll / X 軸



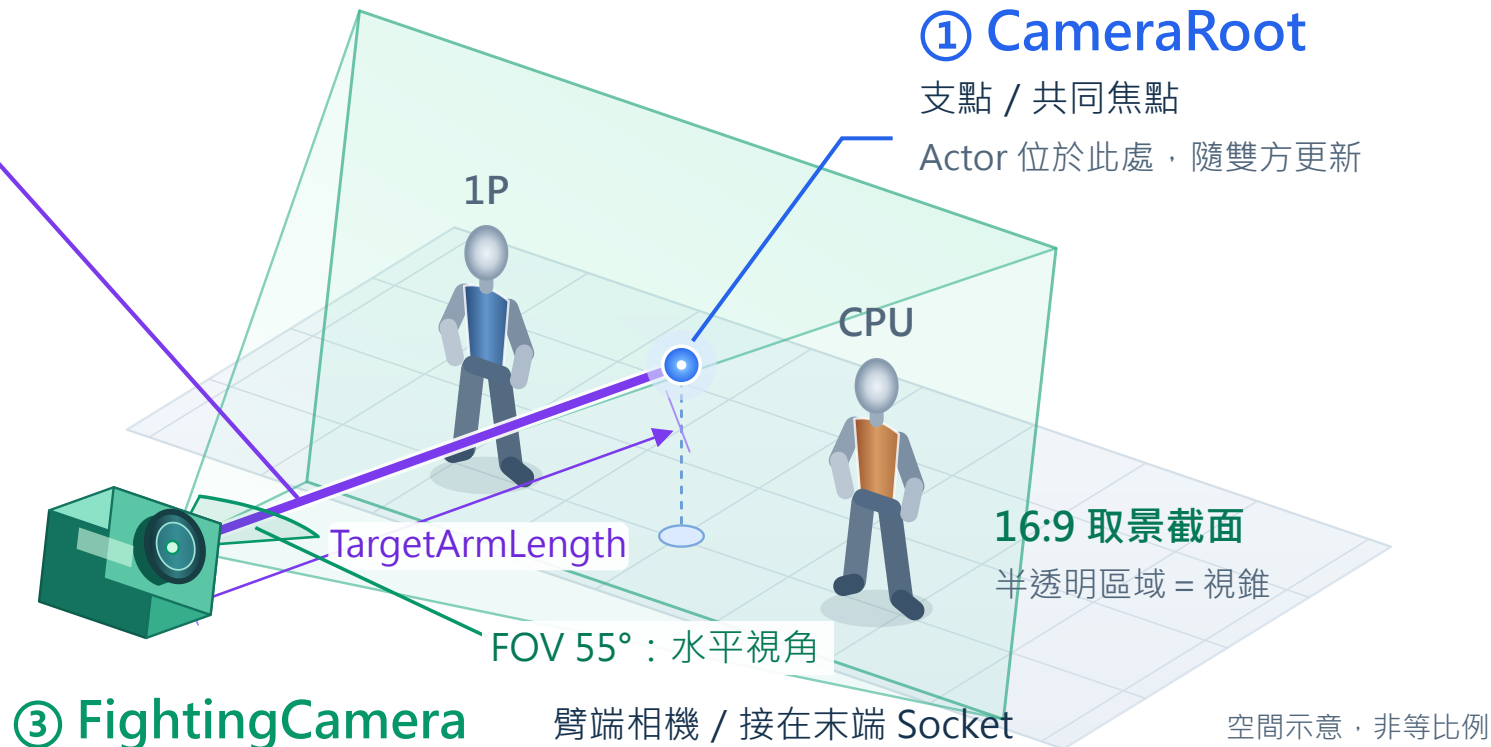
畫面歪斜
預設 0° ，保持不歪斜

Yaw 即時根據兩名角色的水平連線計算，
從側面拍兩人。

① CameraRoot

支點 / 共同焦點

Actor 位於此處，隨雙方更新



③ FightingCamera

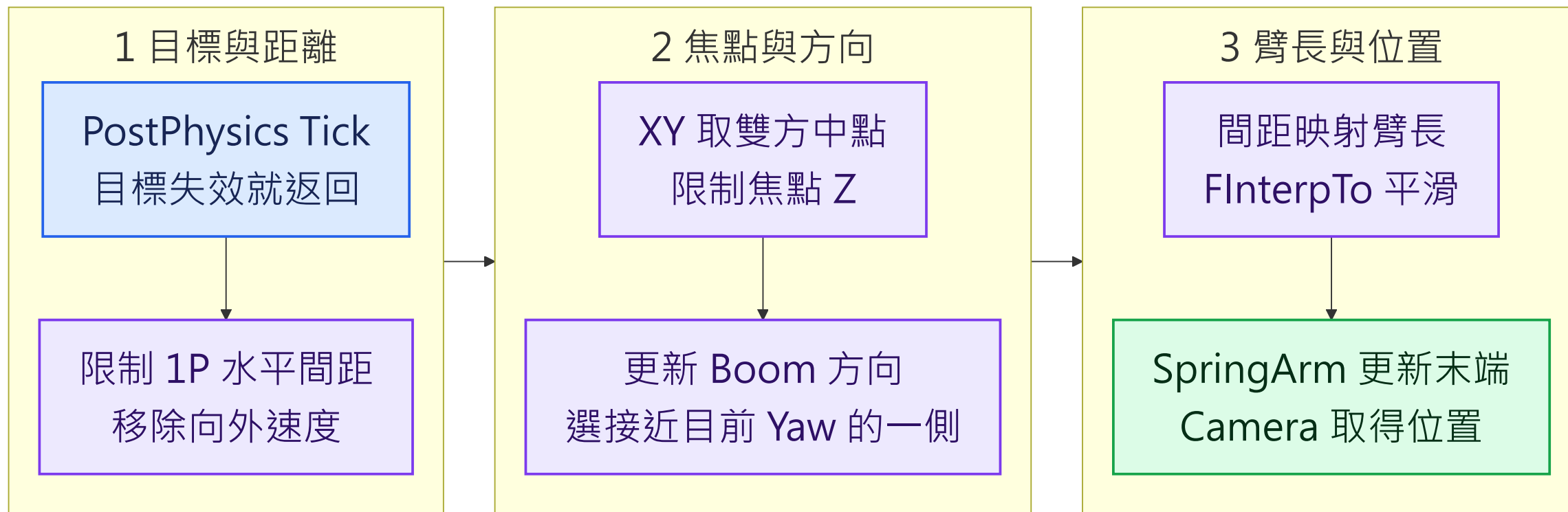
臂端相機 / 接在末端 Socket

空間示意，非等比例



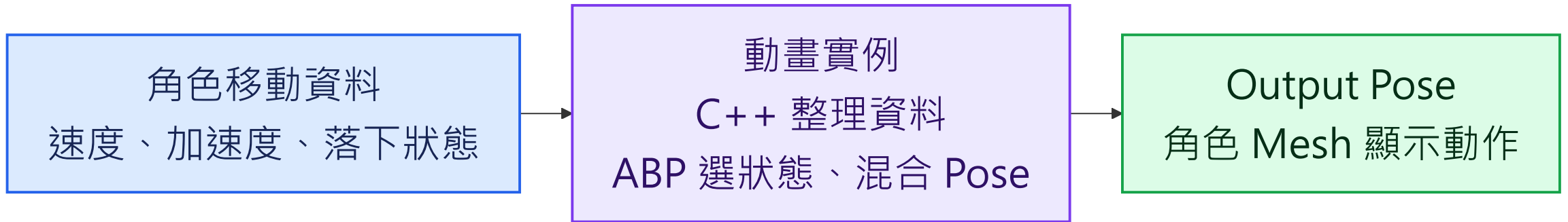
CameraRoot 是支點，CameraBoom 是支臂，FightingCamera 是裝在臂端的相機。

相機跟隨：先限制角色位置，再更新共同取景



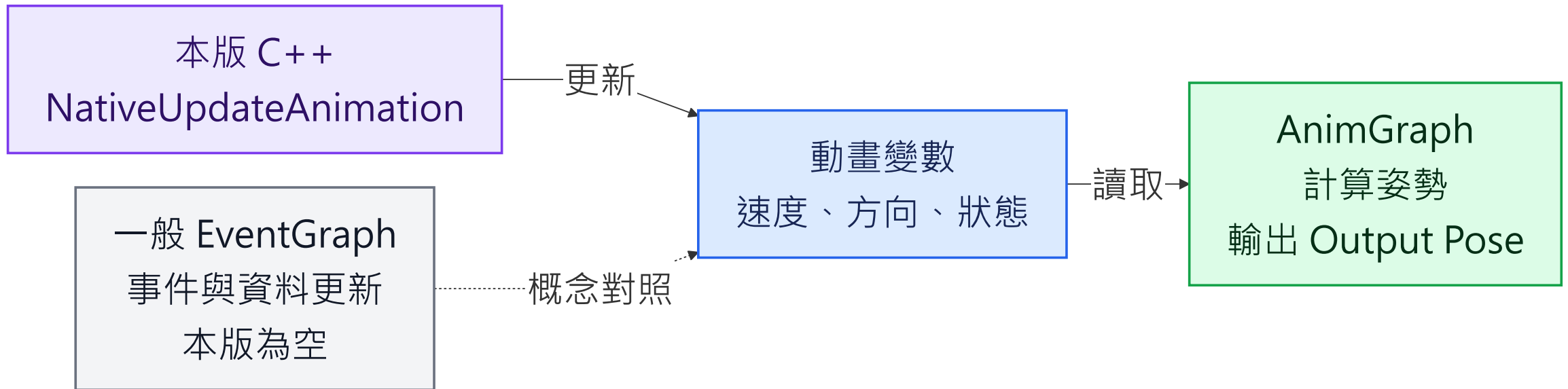
相機使用指定的兩名角色，先限制間距，再算焦點、方向與臂長。

ABP 做什麼：把角色狀態轉成看得見的動作



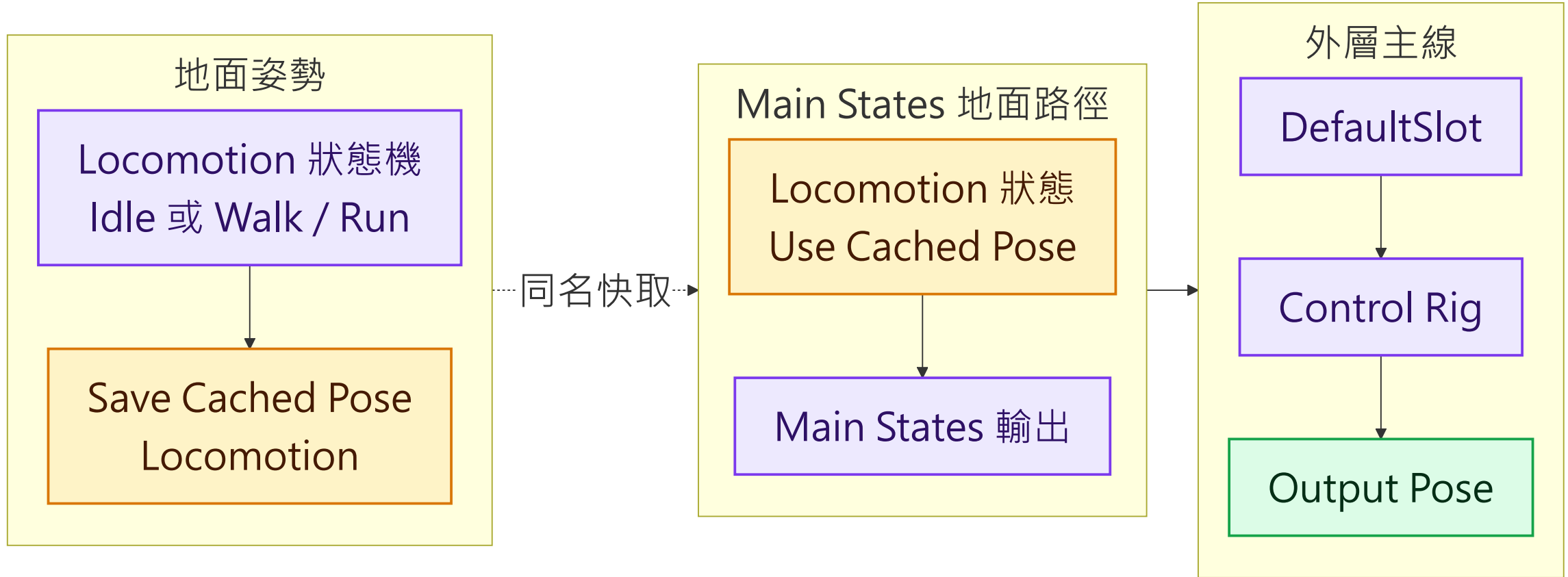
角色提供移動狀態，ABP 依資料選動畫，輸出這次要顯示的骨架姿勢。

EventGraph 與 AnimGraph：更新資料與計算姿勢



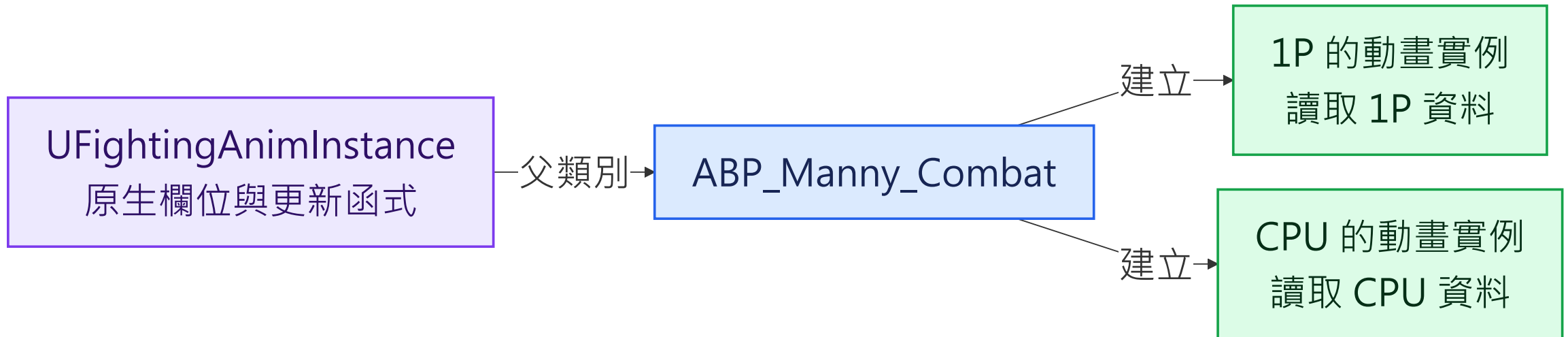
EventGraph 能處理事件與資料，AnimGraph 計算 Pose；本版的資料更新由 C++ 完成。

從 Locomotion 經快取，接到最終 Output Pose



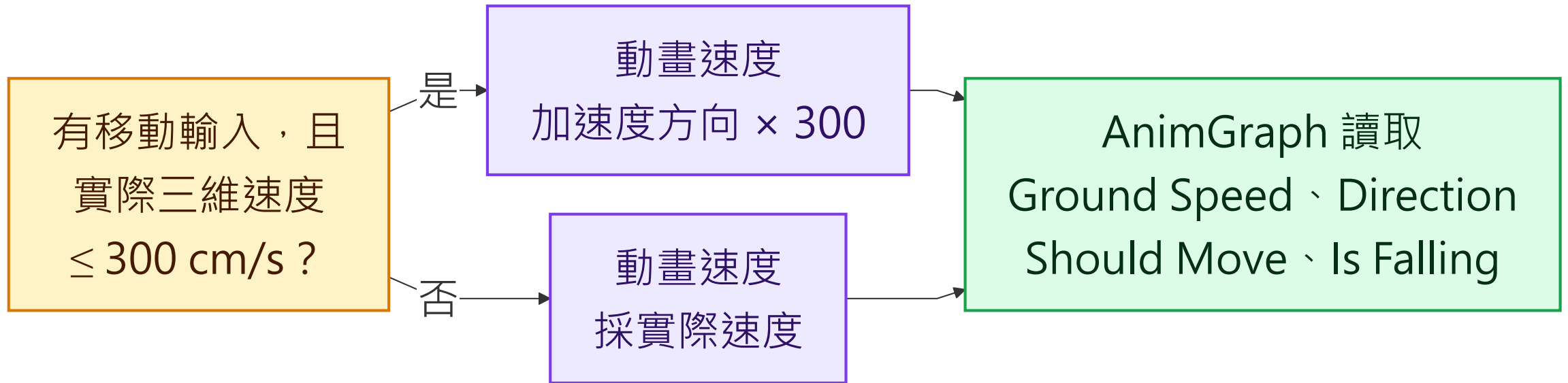
Locomotion 快取姿勢，Main States 取用後，經 Slot、Control Rig 接到 Output Pose。

FightingAnimInstance 與 ABP : 父類別、子類別與執行實例



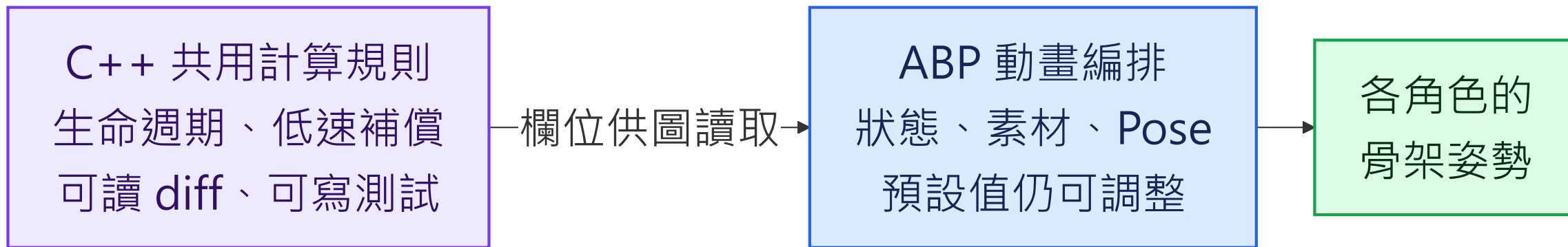
ABP 繼承 FightingAnimInstance，每個角色的動畫實例都帶著父類欄位與自己的數值。

FightingAnimInstance：把真實移動整理成動畫資料



`FightingAnimInstance` 整理動畫用資料，低速仍有輸入時用補償速度維持移動動作。

為什麼用 C++：共用規則可測試，動畫編排留在 ABP



共用的計算規則集中在 C++，狀態、素材與姿勢連線留在 ABP 調整。